CSC 3320: System-Level Programming

Chapter 12: Pointers and Arrays

Instructor: Dr. Md Mahfuzur Rahman
Department of Computer Science, GSU

Using an Array Name as a Pointer

- The two functions below are equivalent

```
int find_largest(int a[], int n)
{
    . . .
}

int find_largest(int *a,  int n)
{
    . . .
}
```

Using an Array Name as a Pointer

- A new version of calculating sum of an array

```
#define N 10
…..
int a[N], i, sum = 0, *p = a;
…..
for(i=0; i< N;i++)
      sum += p[i];
```

Pointers and Multidimensional Arrays

- For a multidimensional array, the name of the array is the address of the first row.
- Example
  - int a[5][5], *p=&a[0][0];
  - To get the element a[0][1], we can use
    - a[0][1], *(a[0]+1), or *(p+1)

```
printf("%p\n", a); // Address of a[0] (first row) Type: int (*)[5]
printf("%p\n", a[0]); // Address of the first element of the first row) Type: int *
printf("%p\n", &a[0]); // Same as a (address of the first row) Type: int (*)[5]
printf("%p\n", &a[0][0]); // Address of first element Type: int *

Note: You may observe the same addresses in output because the address of a row and
address of the first element of that row are essentially the same address.
```

Pointers and Multidimensional Arrays

- Example: Initialize a 2D array

Traditional method

```
int a[NUM_ROWS][NUM_COLS];
int row, col;
for(row = 0; row < NUM_ROWS; row++)
    for(col = 0; col < NUM_COLS; col++)
            a[row][col] = 0;
```

Use pointer

```
int *p;
for(p = &a[0][0]; p <= &a[NUM_ROWS-1][NUM_COLS-1];p++)
        *p = 0;
```

Exercises

- Suppose that the following declarations are in effect:

```
int a[]={5,15,34,54,14,2,52,72}
int *p=&a[1], *q=&a[5];
```

(1) The value of ***(p+3)**? **a[1+3]=14**

(2) The value of ***(q-3)**? **a[5-3]=34**

(3) The value of **q-p**? **5-1=4**

(4) Is **p<q** true of false? **1<5 is true**

(5) Is ***p<*q** true of false? **a[1]<a[5] is false**

Exercises

- What will be the contents of the array **a** after the following statements are executed?

```
int  a[N] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
int *p = &a[0], *q=&a[N-1], temp;

while(p<q){
   temp = *p;
   *p++ = *q;
   *q-- = temp;
}
```

A difference on `const` Position

```
void f(const int *p)
{
   int j;
   *p = 0; /** Wrong - value cannot be modified **/
    p = &j; /*legal*/
}

void fun(int * const p)
{
   int j;
   *p = 0; /* legal */
   p = &j; /*** Wrong - pointer cannot be modified ***/
}
```

Processing the rows

We'll use this simplification in the following loop, which clears row i of the array a:

```
int a[NUM_ROWS][NUM_COLS], *p, i;
for (p = a[i]; p < a[i] + NUM_COLS; p++)
     *p = 0;
```

We can just as easily use find_largest to determine the largest element in row i of the two-dimensional array a:

```
largest = find_largest(a[i], NUM_COLS);
```

Processing the columns

A loop that clears column i of the array a:

```
int a[NUM_ROWS][NUM_COLS], (*p)[NUM_COLS], i;
for (p = &a[0] ; p < &a[NUM_ROWS] ; p++)
    (*p) [i] = 0;
```

- ✓ The expression p++ advances p to the beginning of the next row.
- ✓ In the expression (*p) [i], *p represents an entire row of a, so (*p) [i] selects the element in column i of that row.
- ✓ The parentheses in (*p) [i] are essential, because the compiler would interpret *p [i] as * (p [i]).

Using the Name of a Multidimensional Array as a Pointer

```
int a[NUM_ROWS][NUM_COLS];
```

a is *not* a pointer to a [0] [0]; Instead, it's a pointer to a[0].

to clear column i of the array a, we can write

```
for (p = a; p < a + NUM_ROWS; p++)
(*p) [i] = 0;
```

To do:

- Do the exercises in chapter 11 and 12 of textbook

CSC 3320: System-Level Programming

Chapter 13: Strings

Instructor: Dr. Md Mahfuzur Rahman
Department of Computer Science, GSU

Note: These slides are updated based on original notes from Raj Sunderraman, Michael Weeks, and Yuan Long

Strings

- Java has a String type, but C does not.
- C treats string literals as character arrays.
- A string literal of length `n` will be stored in a `n + 1` bytes memory.
- The last character in the array is the ***null character ('\0')*** as a marker for the end of the string

Do not confuse it with '0'

String Literals

```
char   *str="HelloWorld";
```

- Since a string literal is stored as an array, the compiler treats it (the string literal) as a pointer of type char *.
  - So when `printf("abc")` is called, the address of "`abc`" is passed.
  - So when `printf("%s",str)` is called, the address of "`HelloWorld`" is passed.

- String literals are stored as characters arrays, but you cannot change them.

```
str[0]='h';   /* Error */ /* can be modified if
stored in an array, rather than as a pointer */
```

String Variables

- Any one-dimensional array of characters is used to store strings
- Initializing a string variable:

```
char date1[8]= "June 14";
char date1[] ="June 14";
char date1[8] = {'J', 'u', 'n', 'e', ' ', '1', '4', '\0'};
```

Length could be omitted

  - `date1 is` an array of characters

```
char *str = "June 14";
```

  - `str` is a pointer to a string literal

date1	J	u	n	e		1	4	\0

C String Library

- The C library provides a rich set of functions for performing operations on strings.
- Prototypes for these functions resides in the header `<string.h>`
- To use them, your program must contain the following line:

```
#include <string.h>
```

C String Library

- Use command man to check available string functions in library
- **$man string**

```
char *strcpy(char *dest, const char *src);
```

  - String Copy

```
size_t strlen(const char *s);
```

  - String Length

```
char *strcat(char *dest, const char *src);
```

  - String Concatenate

```
int strcmp(const char *s1, const char *s2);
```

  - String Compare

The *strlen* (String Length) Function

```
size_t strlen(const char *s);
```

- Returns the length of a string s (excluding the first `null` character).
- Example

```
int len = strlen("abc");
/* return 3 */
```

The *strcpy*(String Copy) Function

```
char *strcpy(char *dest,
                    const char *src);
```

- Copies the string `src` into the string `dest`.
- Example

```
str2 = "abcd";

strcpy(str2, "abcd");
strcpy(str1, str2);

strcpy(str1, strcpy(str2, "abcd"));
```

- It is safer to call `strncpy`, but it is slower.

```
char *strncpy(char * dest, const char * src, size_t s);
```

The *strcat*(String Concatenate) Function

```
char *strcat(char *dest,
                        const char *src);
```

- Appends the contents of the string `src` to the end of the string `dest`, and return `dest`
- Example

```
strcpy(str1, "abc");
strcat(str1, "def");
/* str1 now contains "abcdef" */
```

The *strcat*(String Concatenate) Function

- Note: Do not forget how much space is left.
- Example

```
char str1[6] = "abc";
strcat(str1, "def");    /*wrong*/
```

Much safer now!

```
char str1[BUFFER_SIZE];
char *str2="def";
strncpy(s1, s2, BUFFER_SIZE - 1);
strncat(s1, s2, BUFFER_SIZE-strlen(s1)-1);
```